PRACTICAL 3 - BANK

Sepolia PoW Faucet : https://sepolia-faucet.pk910.de/?utm_source=chatgpt.com
Google Cloud Web3 Faucet - https://cloud.google.com/application/web3/faucet/ethereum/sepolia?utm_source=chatgpt.com

CODE:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract BankAccount {

    // Store the balance of each customer
    mapping(address => uint256) private balances;

    // Deposit Ether into the account
    function deposit() public payable {
        require(msg.value > 0, "Deposit amount must be greater than zero");

        balances[msg.sender] += msg.value;
    }

    // Withdraw Ether from the account
    function withdraw(uint256 amount) public {
        require(amount > 0, "Withdrawal amount must be greater than zero");
        require(balances[msg.sender] >= amount, "Insufficient balance");

        // Update balance before transferring Ether
        balances[msg.sender] -= amount;

        // Transfer Ether to the customer
        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Transfer failed");
    }

    // Display the customer's balance
    function showBalance() public view returns (uint256) {
        return balances[msg.sender];
    }

    // Check the contract's total Ether balance
    function contractBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

Part 1 – Basic Solidity statements

```
None
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

- `SPDX-License-Identifier: MIT`: Specifies the open-source license of the code.
- `pragma solidity ^0.8.20`: Specifies the Solidity compiler version. It allows compatible versions from 0.8.20 up to, but not including, 0.9.0.

Part 2 – BankAccount contract

```
None
contract BankAccount {
```

- `contract` is a keyword used to create a smart contract.
- `BankAccount` is the name of our contract.

The purpose of this contract is to allow customers to deposit money, withdraw money and check their balance.

1. Mapping

```
None
mapping(address => uint256) private balances;
```

First, what is mapping?

A mapping stores data in the form of key-value pairs.

Here:

- `address` is the key. It represents the customer's wallet address.
- `uint256` is the value. It stores the customer's balance.
- `balances` is the mapping name.
- `private` means it can be accessed directly only inside this contract.

Example:

Customer address	Balance
0x123...	0.001 ETH
0x456...	0.002 ETH

Each customer has a separate balance.

Why did we use mapping?

Because we want to maintain a separate balance for every customer using their wallet address.

Part 3 – Deposit Function

```
None
function deposit() public payable {
    require(msg.value > 0, "Deposit amount must be greater than zero");

    balances[msg.sender] += msg.value;
}
```

This function is used to deposit Ether into the smart contract.

Let's understand it line by line.

1. Function declaration

```
None
function deposit() public payable
```

- `function`: Creates a function.
- `deposit()`: Name of the function.
- `public`: Can be called from inside and outside the contract.
- `payable`: Allows the function to receive Ether.

Why do we use payable?

Without `payable`, the function cannot accept Ether sent along with the transaction.

2. Require condition

```
None
require(msg.value > 0, "Deposit amount must be greater than
zero");
```

`require()` checks a condition before executing the next statement.

Here, it checks that the deposited amount is greater than zero.

If someone tries to deposit zero ETH, the transaction is reverted.

3. Update balance

```
None
balances[msg.sender] += msg.value;
```

This is the main statement.

- `msg.sender` represents the wallet address calling the function.
- `msg.value` represents the Ether sent with the transaction.
- `balances[msg.sender]` accesses that customer's balance.
- `+=` adds the deposited amount to the existing balance.

Example:

Suppose your current balance is 0 ETH and you deposit 0.001 ETH.

New balance = 0 + 0.001 = 0.001 ETH.

Part 4 – Withdraw Function

```
None
function withdraw(uint256 amount) public {
    require(amount > 0, "Withdrawal amount must be greater than
zero");
    require(balances[msg.sender] >= amount, "Insufficient
balance");
```

```
    balances[msg.sender] -= amount;

    (bool success, ) = payable(msg.sender).call{value:
amount}("");
    require(success, "Transfer failed");
}
```

This function allows a customer to withdraw Ether from the smart contract.

1. Function declaration

```
None
function withdraw(uint256 amount) public
```

- `withdraw` is the function name.
- `uint256 amount` accepts the withdrawal amount.
- `public` allows the function to be called from outside the contract.

For example, we can enter `400000000000000` Wei to withdraw 0.0004 ETH.

2. Check withdrawal amount

```
None
require(amount > 0, "Withdrawal amount must be greater than
zero");
```

Checks that the withdrawal amount is greater than zero.

If the amount is zero, the transaction is rejected.

3. Check available balance

```
None
require(balances[msg.sender] >= amount, "Insufficient balance");
```

Checks whether the customer has enough balance to withdraw.

For example:

- Available balance = 0.001 ETH.
- Withdrawal amount = 0.0004 ETH.

Since the available balance is sufficient, withdrawal is allowed.

If the customer tries to withdraw 0.002 ETH, the transaction is rejected.

4. Deduct balance

```
None
balances[msg.sender] -= amount;
```

Subtracts the withdrawal amount from the customer's recorded balance.

Example:

0.001 − 0.0004 = 0.0006 ETH.

Why do we deduct the balance before transferring Ether?

To prevent security problems such as reentrancy attacks. The contract updates its internal balance before making the external call.

5. Transfer Ether

```
None
(bool success, ) = payable(msg.sender).call{value: amount}("");
```

This line transfers Ether to the customer's wallet.

Let's break it down:

Part	Meaning
`payable(msg.sender)`	Customer's wallet address, converted to a payable address
`.call`	Makes an external call
`{value: amount}`	Sends the specified Ether amount
`""`	Sends empty data

`bool success`	Stores whether the transfer succeeded

What is bool success?

It stores either:

- `true` – Transfer successful.
- `false` – Transfer failed.

6. Check transfer success

```
None
require(success, "Transfer failed");
```

Checks whether the Ether transfer was successful.

If it fails, the transaction is reverted, including the earlier balance deduction.

Part 5 – Show Balance Function

```
None
function showBalance() public view returns (uint256) {
    return balances[msg.sender];
}
```

This function displays the balance of the customer who calls it.

Understand the keywords:

- `public`: Can be called from outside the contract.
- `view`: Reads stored data but does not modify it.
- `returns(uint256)`: Returns an unsigned integer.

```
None
return balances[msg.sender];
```

Returns the balance associated with the caller's wallet address.

Example: If your wallet has deposited 0.001 ETH and withdrawn 0.0004 ETH, it returns 0.0006 ETH.

Part 6 – Contract Balance Function

```
None
function contractBalance() public view returns (uint256) {
    return address(this).balance;
}
```

This function displays the total Ether held by the smart contract.

Let's understand:

- `address(this)` refers to the address of the current smart contract.
- `.balance` returns the Ether balance held at that contract address.
- `view` means it only reads information.
- `returns(uint256)` returns the balance in Wei.

Difference between showBalance() and contractBalance():

showBalance()	**contractBalance()**
Shows the caller's recorded balance.	Shows total Ether held by the contract.
Uses `balances[msg.sender]`.	Uses `address(this).balance`.
Individual customer balance.	Overall contract balance.

For example, if two customers have deposited 0.001 ETH and 0.002 ETH respectively:

- Customer 1's `showBalance()` = 0.001 ETH.
- Customer 2's `showBalance()` = 0.002 ETH.
- `contractBalance()` = 0.003 ETH.

Part 7 – Complete practical dry run

Let's understand how we performed this practical.

Step 1 – Deploy contract

Compile the BankAccount contract in Remix and deploy it using MetaMask on Sepolia.

Step 2 – Deposit money

Deposit amount
0.001 ETH

The `deposit()` function adds the amount to the customer's recorded balance.

Step 3 – Check balance

`showBalance()`
0.001 ETH

Step 4 – Withdraw money

Withdrawal amount
0.0004 ETH

The `withdraw()` function checks the balance, deducts the amount and transfers Ether.

Step 5 – Check balance again

`showBalance()`

0.0006 ETH

Step 6 – Check contract balance

`contractBalance()` displays the total Ether held by the contract.

The example assumes no other deposits or withdrawals and ignores gas fees paid separately by the wallet.

Part 8 – Important Solidity keywords

Keyword	Simple meaning
`mapping`	Stores key-value pairs

`address`	Stores a wallet or contract address
`uint256`	Stores non-negative integers
`public`	Accessible from inside and outside
`private`	Accessible only inside the same contract
`payable`	Allows receiving Ether
`require`	Checks a condition
`msg.sender`	Address of the caller
`msg.value`	Amount of Ether sent
`view`	Reads data without changing it
`returns`	Specifies the returned value
`call`	Makes an external call
`bool`	Stores true or false

`this`	Refers to the current contract

Part 9 – Important viva questions

Q1. Why did we use mapping?

To store the balance of each customer using their wallet address.

Q2. Why is deposit() payable?

Because it needs to accept Ether from the customer.

Q3. What is msg.sender?

It represents the wallet address that calls the function.

Q4. What is msg.value?

It represents the amount of Ether sent with the current transaction, in Wei.

Q5. Why do we use require()?

To check conditions and prevent invalid operations.

Q6. Why do we deduct balance before transferring Ether?

To reduce the risk of reentrancy attacks.

Q7. What is the difference between showBalance() and contractBalance()?

showBalance() displays the caller's recorded balance, whereas contractBalance() displays the total Ether held by the contract.

Q8. What happens if someone withdraws more than their balance?

The transaction is reverted with the message "Insufficient balance".

Q9. Why do we use call() for transferring Ether?

It allows us to send Ether to the customer's address and check whether the transfer succeeded.

Q10. What is the use of bool success?

It tells us whether the Ether transfer was successful or failed.

Q11. What is the difference between ETH and Wei?

ETH is the main currency unit, while Wei is its smallest unit.

1 ETH = 10^{18} Wei.

Q12. What is a smart contract?

A program deployed on a blockchain that executes predefined instructions.

PR 3: Bank Account Smart Contract Using Solidity on Sepolia Testnet

Aim

To write, compile and deploy a Bank Account smart contract on a test network that performs the following operations:

1. Deposit money.
2. Withdraw money.
3. Show the customer's balance.

Software and Tools Required

- Remix IDE: https://remix.ethereum.org/
- MetaMask browser extension.
- Sepolia Testnet.
- Free Sepolia test ETH from a faucet.

Important: We will use Sepolia test ETH, not real money. Never purchase Mainnet ETH for this practical.

Step 1: Install and Set Up MetaMask

1. Open Chrome and install the MetaMask extension from its official website: https://metamask.io/
2. Create a new wallet or import an existing wallet.
3. Set a password.
4. Securely save your Secret Recovery Phrase. Never share it with anyone.
5. Open MetaMask.
6. Go to Settings → Networks.
7. Enable **Show test networks**.
8. Select **Sepolia Test Network**.

Step 2: Get Free Sepolia Test ETH

1. Open the Google Cloud Sepolia Faucet.
2. Connect or enter your MetaMask wallet address as requested.
3. Request free Sepolia test ETH.
4. Wait for the transaction to complete.
5. Open MetaMask and verify that your SepoliaETH balance has increased.

You need test ETH to pay for deployment, deposit and withdrawal transaction gas fees.

Step 3: Open Remix IDE

1. Visit https://remix.ethereum.org/
2. Create a new Solidity file:
    - Open File Explorer.
    - Create a new file named `BankAccount.sol`.
3. Open the file and write the following Solidity code.

Step 4: Write the Smart Contract

None

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract BankAccount {

    // Store the balance of each customer
    mapping(address => uint256) private balances;

    // Deposit Ether into the account
    function deposit() public payable {
        require(msg.value > 0, "Deposit amount must be greater
than zero");

        balances[msg.sender] += msg.value;
    }

    // Withdraw Ether from the account
    function withdraw(uint256 amount) public {
        require(amount > 0, "Withdrawal amount must be greater
than zero");
```

```
        require(balances[msg.sender] >= amount, "Insufficient
balance");

        // Update balance before transferring Ether
        balances[msg.sender] -= amount;

        // Transfer Ether to the customer
        (bool success, ) = payable(msg.sender).call{value:
amount}("");
        require(success, "Transfer failed");
    }

    // Display the customer's balance
    function showBalance() public view returns (uint256) {
        return balances[msg.sender];
    }

    // Check the contract's total Ether balance
    function contractBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

Step 5: Compile the Smart Contract

1. Open the Solidity Compiler tab from the left sidebar.
2. Select compiler version `0.8.20` or a compatible 0.8.x version.
3. Open Advanced Compiler Configurations.
4. Set **EVM Version = Paris**.
5. Click **Compile BankAccount.sol**.
6. A green tick should appear, indicating successful compilation.

Why select Paris?

It avoids compatibility issues with older EVM environments that may not support newer instructions generated by the compiler.

Step 6: Connect Remix to MetaMask

1. Open the **Deploy & Run Transactions** tab from the left sidebar.
2. Under Environment, select **Browser Extension**.

3. In the next dropdown, select **Sepolia Testnet - MetaMask**.
4. MetaMask will ask for permission to connect.
5. Select your wallet account and click Connect.
6. Verify that Remix displays:
    - Environment: Browser Extension
    - Network: Sepolia Testnet - MetaMask
    - Your connected MetaMask account and test ETH balance.

Important: Do not use Remix VM if you want to deploy on the public Sepolia test network. Remix VM is only a local simulated blockchain.

Step 7: Deploy the Smart Contract

1. In Deploy & Run Transactions, open the **Deploy** tab.
2. Under Contract, select `BankAccount`.
3. Set the Value field to `0`.
4. Keep the unit as `wei`.
5. Click **Deploy**.
6. MetaMask will display a deployment confirmation.
7. Verify that the network is Sepolia.
8. Check the network fee and click **Confirm**.
9. Wait for the transaction to complete.

After successful deployment:

- Open the Deployed Contracts tab.
- You should see `BankAccount`.
- Expand it to view the available functions.

The contract is now deployed on Sepolia Testnet.

Step 8: Deposit Money

We will deposit 0.001 ETH into the customer's bank account.

1. Under Deployed Contracts, find the `deposit()` function.
2. Scroll down in the Deploy & Run panel to find the **Value** field.
3. Enter `1000000000000000`.
4. Keep the unit as `wei`.

Why this number?

1 ETH = 10^{18} Wei

0.001 ETH = 10^{15} Wei

5. Scroll up and click **Transact** beside `deposit()`.

6. MetaMask will open.
7. Verify:
    - Network: Sepolia.
    - Amount: 0.001 SepoliaETH.
8. Click Confirm.
9. Wait for the transaction to succeed.

The deposit is now complete.

Note: The deposit function is payable, so the ETH amount must be entered in the Remix Value field, not in a function argument.

Step 9: Display the Customer's Balance

1. Find `showBalance()` under Deployed Contracts.
2. Click **Call**.
3. Remix displays the customer's balance in Wei.

Expected output:

```
1000000000000000 Wei
```

This is equal to 0.001 ETH.

Step 10: Check the Contract's Total Balance

1. Find `contractBalance()`.
2. Click **Call**.
3. Remix displays the total ETH held by the smart contract.

Expected output:

```
1000000000000000 Wei
```

This is also equal to 0.001 ETH.

Difference between the two functions:

- `showBalance()` displays the balance of the currently connected customer.
- `contractBalance()` displays the total Ether held by the smart contract.

Step 11: Withdraw Money

Now we will withdraw 0.0004 ETH.

1. Find `withdraw(uint256)` under Deployed Contracts.
2. Enter the withdrawal amount in the `amount` field.

Enter:

`400000000000000`

This is equal to 0.0004 ETH in Wei.

3. Before clicking Transact, scroll down to the Remix **Value** field.
4. Make sure Value is `0 wei`.
5. Scroll back up.
6. Click **Transact** beside `withdraw`.
7. MetaMask will open.
8. Verify the Sepolia network and confirm the transaction.
9. Wait for successful execution.

Important: The withdrawal function is not payable. Therefore, do not send Ether through the Value field while withdrawing.

Step 12: Verify the Final Balance

After successful withdrawal, call both functions again:

- `showBalance()`
- `contractBalance()`

Expected output:

`600000000000000 Wei`

This is equal to 0.0006 ETH.

Final Calculation

Initial deposit = 0.001 ETH

Withdrawal = 0.0004 ETH

Remaining balance:

0.001 − 0.0004 = 0.0006 ETH

Observation Table

Operation	Amount	Result
Deploy contract	0 ETH	Successful

Deposit	0.001 ETH	Successful
Show balance	0.001 ETH	Successful
Contract balance	0.001 ETH	Successful
Withdraw	0.0004 ETH	Successful
Final balance	0.0006 ETH	Successful

Important Concepts Used

1. Smart Contract: A program deployed on a blockchain that executes automatically when its conditions are satisfied.

2. Solidity: A programming language used to develop Ethereum smart contracts.

3. Mapping: Stores key-value pairs. Here, the customer's wallet address is mapped to their balance.

4. `msg.sender`: Represents the address of the account calling the function.

5. `msg.value`: Represents the amount of Ether sent with a payable transaction, measured in Wei.

6. `payable`: Allows a function to receive Ether.

7. `require()`: Checks conditions and reverts the transaction if a condition is false.

8. `uint256`: An unsigned 256-bit integer used to store the balance and withdrawal amount.

9. `view`: Indicates that a function reads contract data without modifying blockchain state.

10. `address(this).balance`: Returns the total Ether balance held by the smart contract.

11. Wei: The smallest denomination of Ether. 1 ETH = 10^{18} Wei.

Common Errors and Their Solutions

Error	Reason	Solution

Invalid opcode during deployment	EVM/compiler version mismatch	Set EVM Version to Paris and recompile
Cannot convert decimal to BigInt	`uint256` expects an integer	Enter the amount in Wei
Insufficient balance	Customer has not deposited enough	Deposit first and check `showBalance()`
Withdrawal transaction fails	Non-zero Value sent with withdrawal	Set Remix Value to 0 Wei
No contracts deployed	Contract deployed only on Remix VM or wrong environment	Connect to Sepolia and deploy again
Balance shows zero	Wrong account or wrong deployment selected	Check connected account and deployed contract

Result

Thus, the Bank Account smart contract was successfully written in Solidity, compiled and deployed on the Sepolia Testnet using Remix IDE and MetaMask.

The deposit, withdrawal, customer balance display and contract balance operations were successfully performed and verified.